Documento de Arquitectura de Infraestructura (HLD)

Microservicio de Almacenamiento de Evidencias

Arquitectura de despliegue, topología de red y portabilidad hacia Azure Cloud.

Visión General del Sistema

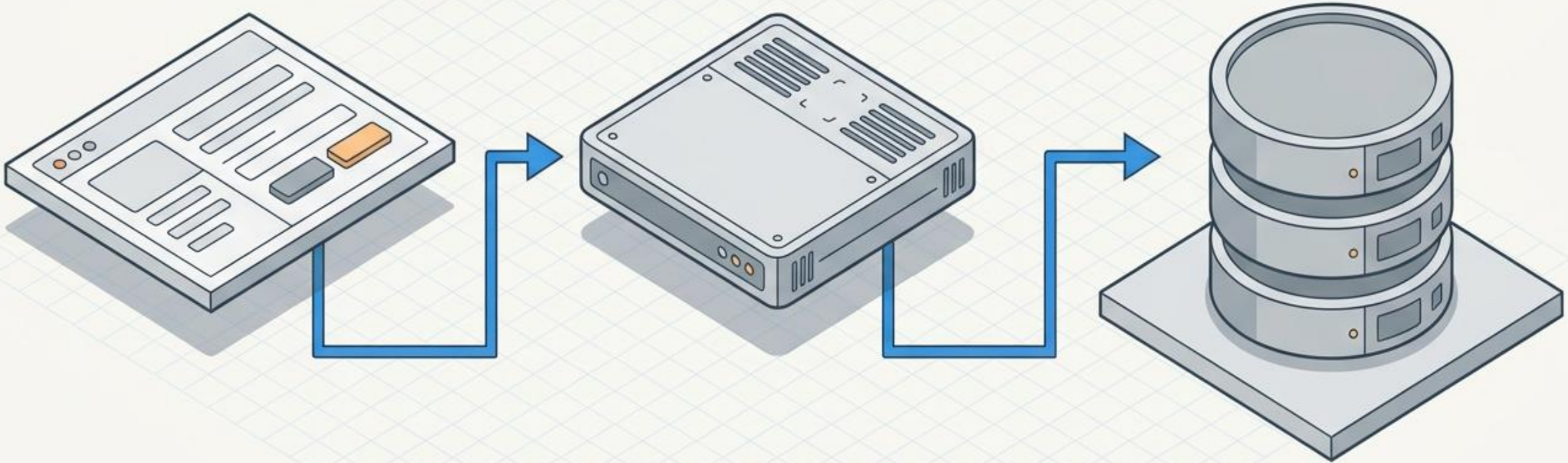
Orquestado íntegramente vía Docker Compose

Solución robusta para gestionar datos persistentes separando estrictamente el cómputo del almacenamiento.

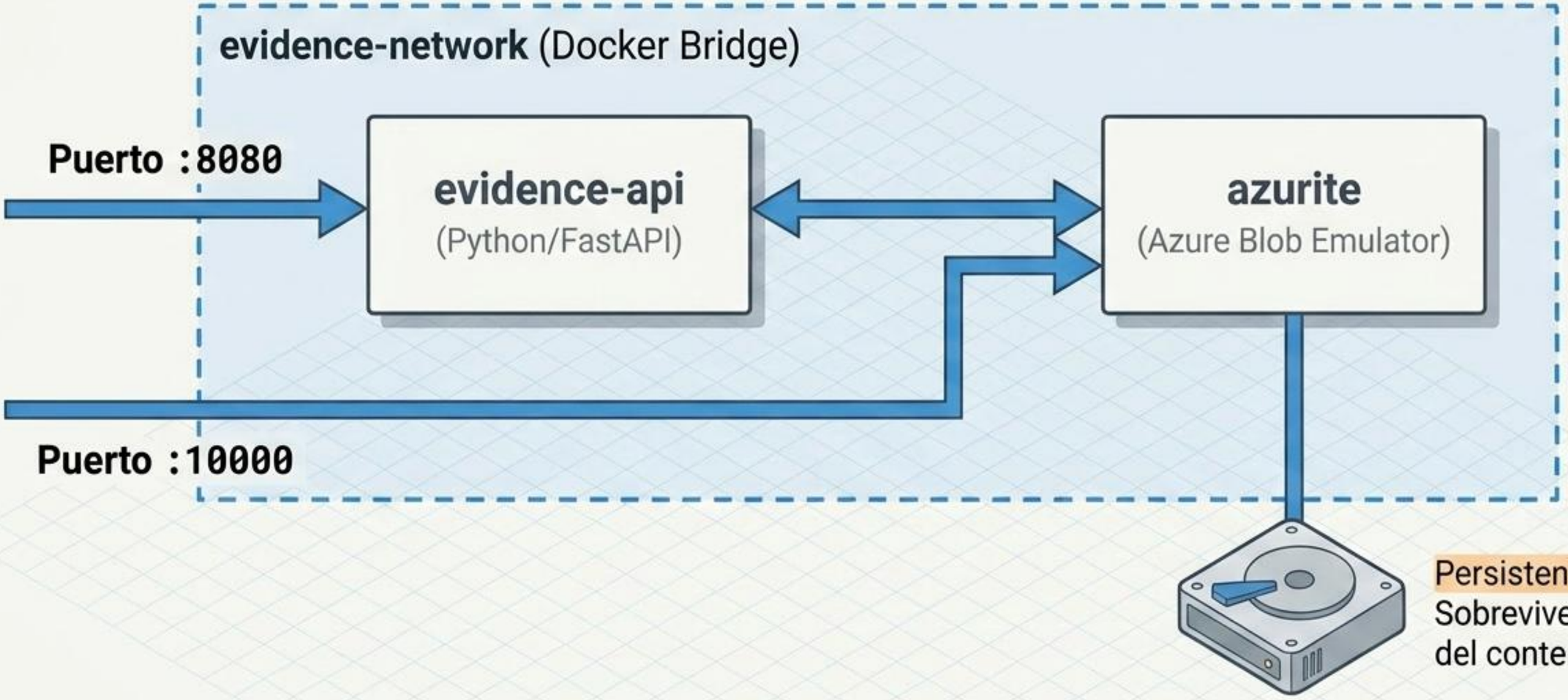
Capa de Cliente
Origen de las peticiones

Cómputo (Stateless)
API REST (FastAPI)

Persistencia (Stateful)
Almacenamiento Blob (Azurite/Azure)



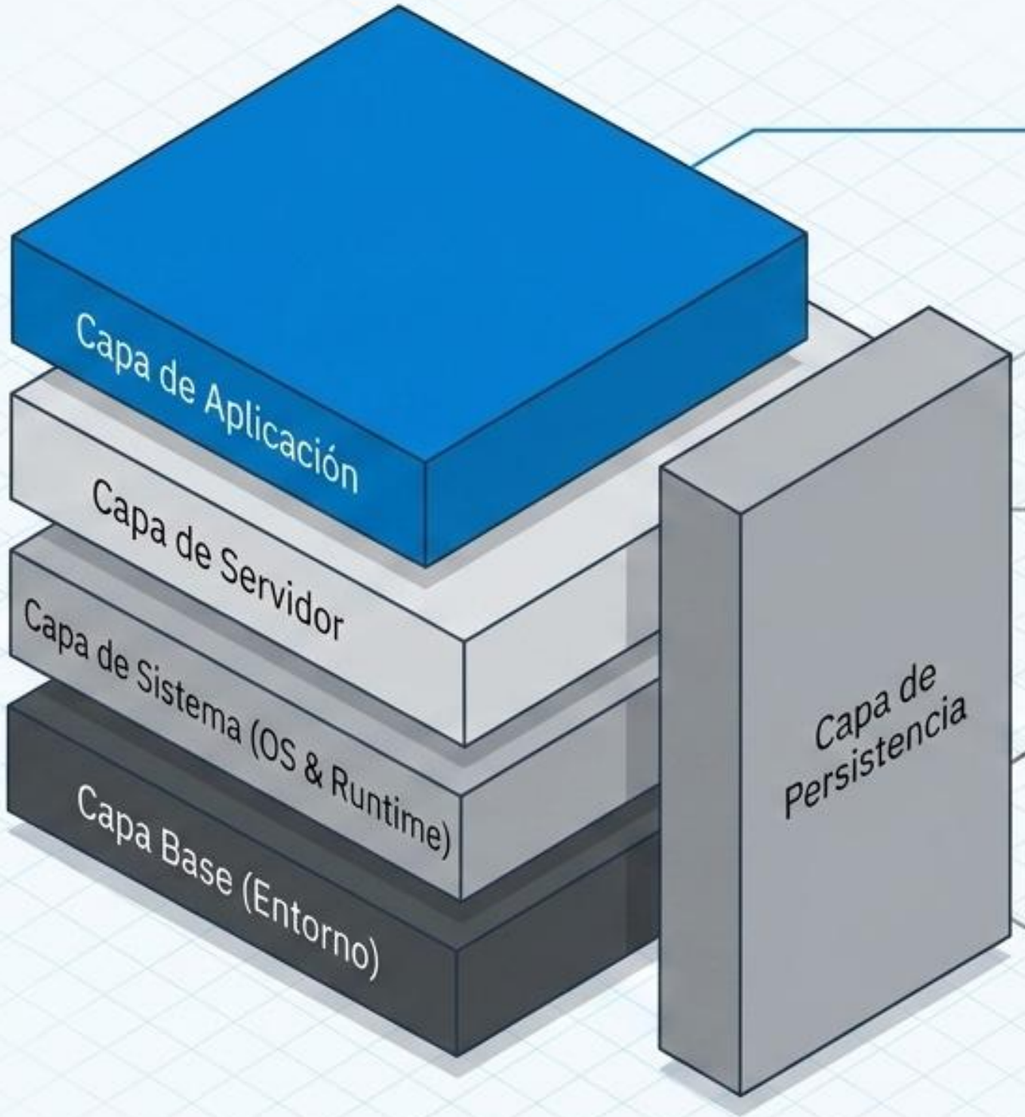
Topología de Red y Aislamiento



Persistencia física. Sobrevive a reinicios del contenedor.

Volumes: azurite-data

Stack Tecnológico (Modelo de Capas)



FastAPI 0.109+ / Pydantic 2.5+

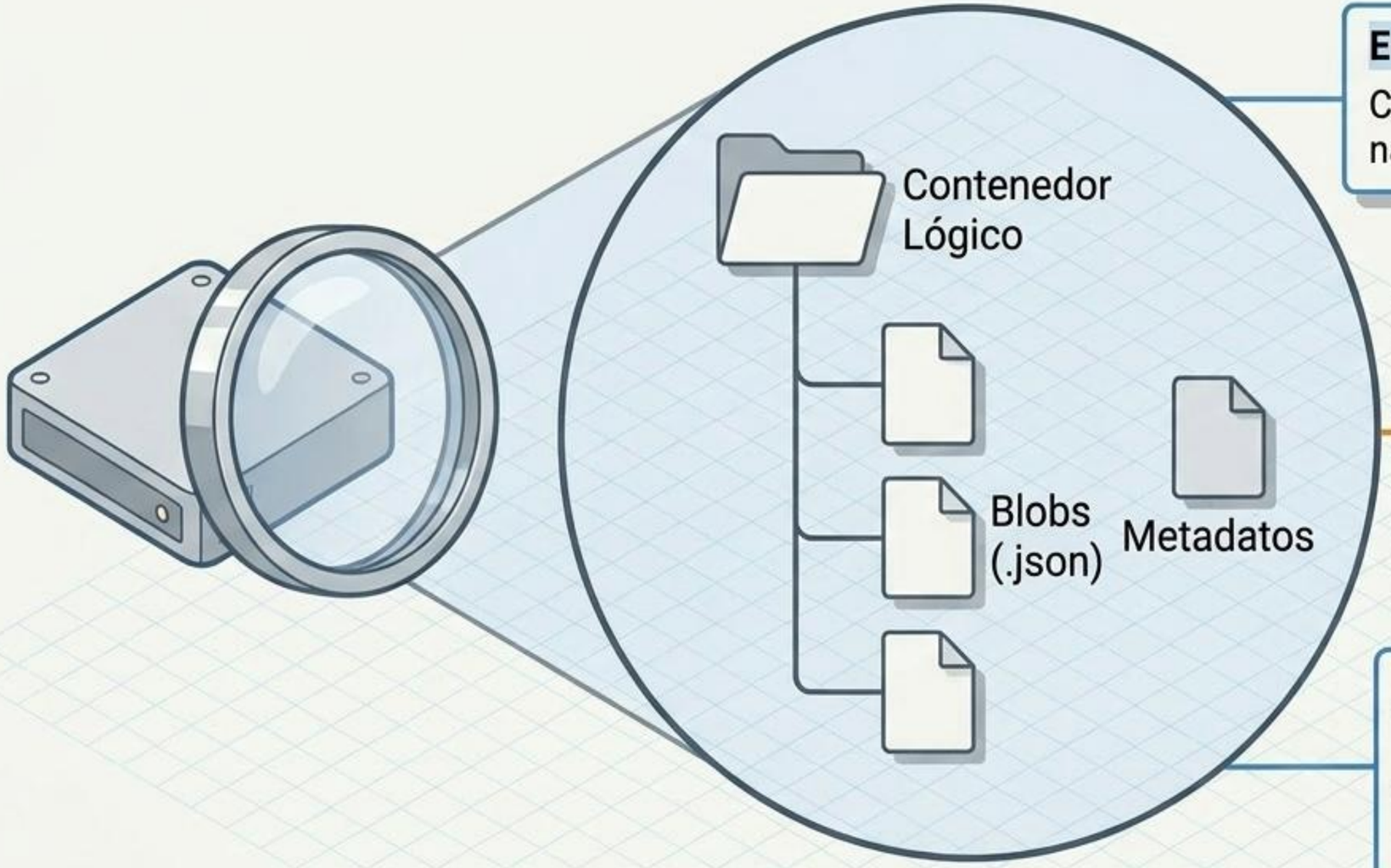
Uvicorn 0.27+ (ASGI, alta concurrencia asíncrona)

Python 3.11-slim (Optimización de peso de imagen)

Docker Engine 20.10+ / Compose 3.8

Azurite 3.35+ / azure-storage-blob 12.19.0

Motor de Persistencia: Azurite



Emulador Oficial

Corre Azure Storage de forma nativa en entorno local.

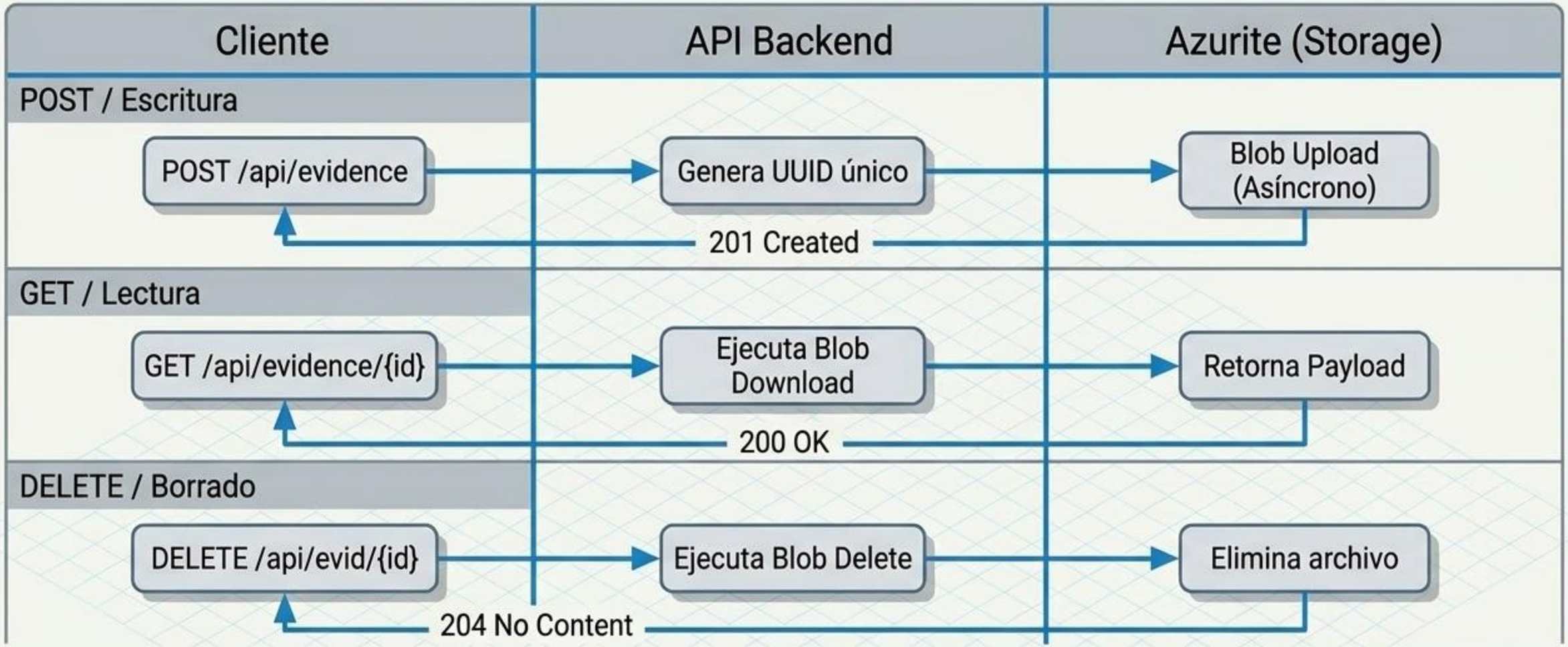
Compatibilidad Crítica

Forzado de `API Version` "`2023-08-03`" y uso del flag "`--skipApiVersionCheck`" para garantizar tolerancia a librerías de SDK modernas.

Autenticación y Secretos

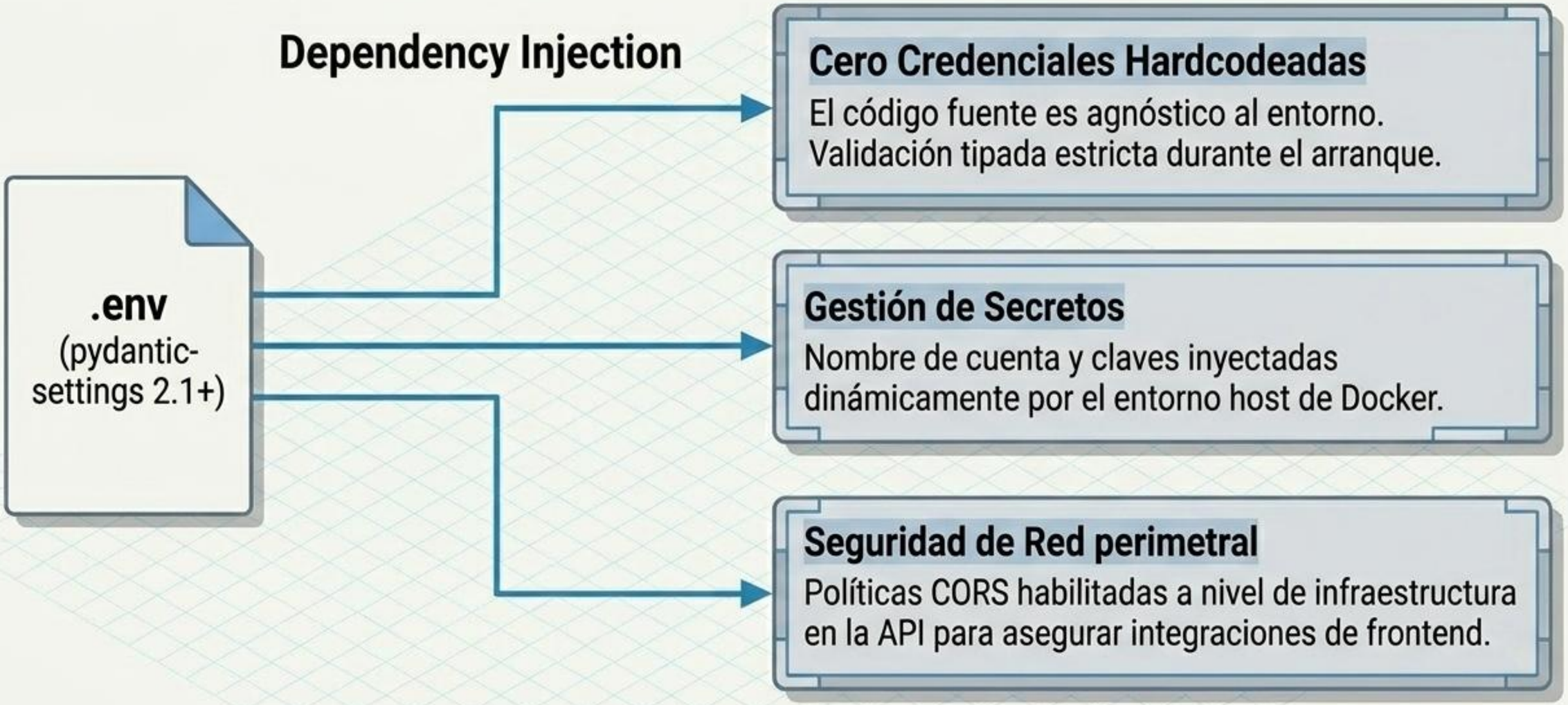
Inyección de cuentas de servicio en tiempo de despliegue mediante la variable `AZURITE_ACCOUNTS`.

Enrutamiento y Flujo de Tráfico



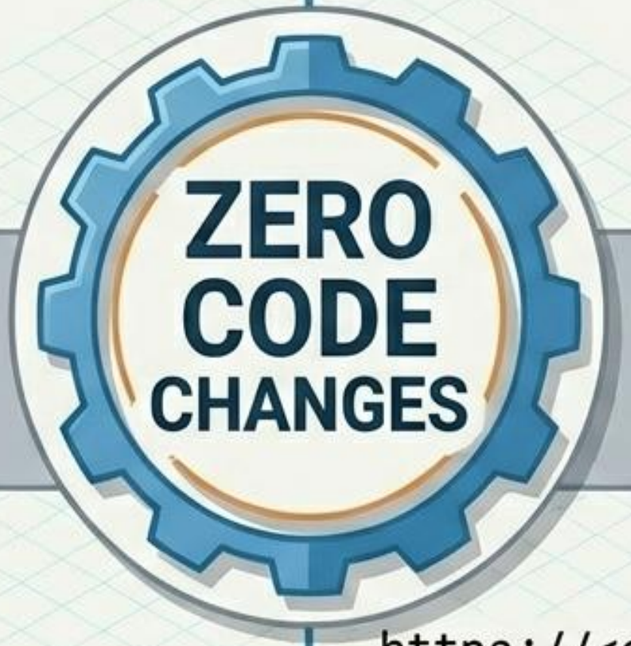
Nota Operativa: El evento 'lifespan' del backend inicializa y valida la conexión al almacenamiento antes de abrir puertos de red.

Configuración Dinámica y Seguridad



Matriz de Migración: Local vs. Nube

Desarrollo Local (Azurite)	Producción Cloud (Azure Blob)
ACCOUNT_NAME: `devstoreaccount`	ACCOUNT_NAME: `<azure_account_name>`
ACCOUNT_KEY: `(default azurite key)`	ACCOUNT_KEY: `<azure_storage_key>`
ENDPOINT: `http://azurite:10000/...`	ENDPOINT: `https://<account>.blob.core.windows.net`



Conclusión: Migrar a producción es puramente una operación de DevOps. El SDK de Azure en la aplicación maneja ambos entornos de forma nativa modificando únicamente el archivo de entorno.

Síntesis de Diseño Arquitectónico